

Trabalho Prático – Processamento Gráfico: Fundamentos

JOGO 2D SIMPLES EXPLORANDO CAMADAS E ANIMAÇÃO POR SPRITES



Figura 1. Uma boa fonte de inspiração são os jogos de Arcade das décadas de 70 e 80. Fonte: Imagem da série Stranger Things (Netflix)



Entrega

Individual ou grupos de até 3 participantes

DATA DE ENTREGA E APRESENTAÇÃO: até 07/10/2026, via Moodle

Instruções para envio do trabalho: conforme o documento *Instruções para Entrega dos Trabalhos*, no Moodle.

Objetivo

O objetivo deste trabalho é desenvolver um protótipo de jogo 2D, utilizando a API Gráfica OpenGL (moderna), aplicando os conceitos vistos em aula. Para o Grau A, os conhecimentos sobre *desenho de primitivas*, transformações geométricas, projeção ortográfica e mapeamento de textura serão explorados.

Instruções de desenvolvimento

O programa deve possuir, como requisitos mínimos:

- Utilizar a API na versão 3.3 ou superior, o que caracteriza que é obrigatória a implementação de **shaders**.
- Correta criação e utilização correta dos buffers de geometria
 - VAO, VBO(s) e EBO (este último é opcional)

- Correta utilização das **transformações** de projeção (*projection matrix*) e nos objetos (*model matrix*)
 - Como está mapeada a matriz de projeção ortográfica?
 - Como são feitas e atualizadas as transformações nos objetos da cena?
- **Sprites** como textura mapeadas em polígonos, para o desenho de:
 - personagem, objetos e itens animados (utilizando *spritesheets*)
 - imagens de fundo (uso de camadas)
 - fundos multi-camadas podemos implementar o conceito de *parallax scrolling*
- Controle da movimentação do personagem (com input de teclado e/ou mouse)
 - Colisão entre os *sprites* (conceito de *hitboxes*), para ações como coletar itens, tiros etc., conforme a proposta do jogo)

É recomendado que o jogo tenha um arquivo de configuração, para carregar imagens das camadas e demais configurações necessárias.

SUGESTÕES DE JOGOS

A escolha do tipo de jogo está a critério do aluno. Se estiverem sem nenhuma “inspiração”, seguem algumas ideias que se encaixam perfeitamente em nosso escopo:

- Talvez o jogo mais simples que contenha tudo que precisamos é daqueles em que caem objetos (por exemplo, de cima pra baixo) e você precisa pegá-los movimentando um outro objeto (por exemplo, esquerda e direita). O primeiro jogo nessa linha foi o Catch do Atari (1977), mas existem milhares de jogos similares, normalmente com maçãs caindo.
- Jogos do tipo endless runner com visão lateral também se enquadram bem no escopo, como por exemplo o do dino do Google
- Jogos antigos de atari ou jogos próprios para fliperama mais antigos são ótimas pedidas também. Você pode se inspirar em sites como <https://www.retrogames.cz/index.php>

EXTRAS (opcionais):

- HUDs com texto (usando, por exemplo, a [FreeType](#))
- Telas de entrada, de jogo e *gameover*
- Som (SDL mixer, OpenAL, FMOD etc)

Apresentação e entrega

- **A apresentação será feita em aula (presencial).** É importante e de grande peso na avaliação que a dupla esteja sincronizada e entenda toda a estruturação do código (classes, decisões de implementação). Pontos que devem ser abordados na apresentação:
 - Apresentação do grupo (nomes)
 - Estrutura geral do código (classes e/ou funções)

- o Estrutura dos buffers e shader(s) (como você especificou os vértices e seus atributos e como você os envia para os shaders)
 - o Gerenciamento dos *sprites*: como são armazenadas as informações das transformações, ID de texturas, infos adicionais
 - o Como é feito o controle do personagem (teclado e/ou mouse)
 - o Como são verificadas as colisões
 - o Como é feita a movimentação de outros objetos do cenário
 - o Como é gerenciada a animação dos sprites
 - o Como são gerenciadas as camadas para o efeito de *Parallax*
 - o Mostrar o programa em funcionamento (sugestão: pode fazer isso no início, logo após a apresentação). Se julgar necessário, pode mostrar em outros momentos junto com a explicação.
- **COMUNICAÇÃO E COLABORAÇÃO SÃO AS CHAVES PARA O SUCESSO.** PROIBIDO FICAR TRANCADO COM DÚVIDAS!

BOM TRABALHO! 😊

Lembre-se que todo problema maior pode ser decomposto em problemas menores

